

Assignment 21.2

Shruthi Addagudi | 2303A52133 | Batch 37

Task 1 – AI-Assisted Pair Programming

Objective

To understand how AI can assist in generating, improving, and analyzing code.

Task 1 – AI-Assisted Pair Programming (Single Prompt)

Act as an AI pair programmer.

1. First, generate a simple To-Do List Manager program in [Python/Java/JavaScript] with basic features:

- Add tasks
- View tasks
- Delete tasks
- Use beginner-friendly logic and comments

2. Then, analyze the code and suggest improvements including:

- Better error handling
- Additional useful features
- Code refactoring and optimization
- Improved user experience

3. Provide an improved version of the program with these enhancements.

4. Finally, compare the original and improved versions by explaining:

- Key changes made
- Why they improve the program
- How AI contributes to better coding and collaboration

Initial Code Generated Using AI

C: > Users > lenovo > Desktop > 3-2 > AIAC > Code 9.py > ...

```
2 # A beginner-friendly command-line to-do list manager
3
4 tasks = []
5
6 def show_menu():
7     print("\n=== To-Do List Manager ===")
8     print("1. Add task")
9     print("2. View tasks")
10    print("3. Delete task")
11    print("4. Quit")
12
13 def add_task():
14     task = input("Enter a task to add: ").strip()
15     if task:
16         tasks.append(task)
17         print(f"Added: {task}")
18     else:
19         print("Empty task not added.")
20
21 def view_tasks():
22     if not tasks:
23         print("No tasks yet.")
24         return
25     print("\nYour tasks:")
26     for i, task in enumerate(tasks, start=1):
27         print(f"{i}. {task}")
28
29 def delete_task():
30     if not tasks:
31         print("No tasks to delete.")
32         return
33     view_tasks()
34     idx = input("Enter the task number to delete: ").strip()
35     if idx.isdigit():
```

0 ▲ 0 Update is ready, click to restart

Spaces: 4 UTF-8 Python

C: > Users > lenovo > Desktop > 3-2 > AIAC > Code 9.py > ...

```
29 def delete_task():
30     view_tasks()
31     idx = input("Enter the task number to delete: ").strip()
32     if idx.isdigit():
33         idx = int(idx)
34         if 1 <= idx <= len(tasks):
35             removed = tasks.pop(idx - 1)
36             print(f"Deleted: {removed}")
37         else:
38             print("Invalid task number.")
39     else:
40         print("Please enter a number.")
41
42 def main():
43     while True:
44         show_menu()
45         choice = input("Choose an option: ").strip()
46
47         if choice == "1":
48             add_task()
49         elif choice == "2":
50             view_tasks()
51         elif choice == "3":
52             delete_task()
53         elif choice == "4":
54             print("Goodbye!")
55             break
56         else:
57             print("Invalid option. Try again.")
58
59 if __name__ == "__main__":
60     main()
```

0 ▲ 0 Update is ready, click to restart

Spaces: 4 UTF-8 Python

```
Your tasks:
1. complete work

=== To-Do List Manager ===
1. Add task
3. Delete task
4. Quit
Choose an option: 2

Your tasks:
1. complete work

=== To-Do List Manager ===
1. Add task
1. complete work
```

AI-Suggested Improvements

The AI suggested the following enhancements:

- Added error handling for invalid inputs
- Improved user interaction
- Added extra features (e.g., mark task complete, edit task)
- Refactored code for better readability

Improved Code

C: > Users > lenovo > Desktop > 3-2 > AIAC > Code 9.py > TodoManager

```
1  # improved_todo.py
2  import json
3  import os
4  from datetime import datetime
5
6  DATA_FILE = "todo_data.json"
7
8  class Task:
9      def __init__(self, text, done=False, created_at=None):
10         self.text = text
11         self.done = done
12         self.created_at = created_at or datetime.now().isoformat()
13
14         def to_dict(self):
15             return {"text": self.text, "done": self.done, "created_at": self.created_at}
16
17         @staticmethod
18         def from_dict(data):
19             return Task(data["text"], data.get("done", False), data.get("created_at"))
20
21     class TodoManager:
22         def __init__(self, data_file=DATA_FILE):
23             self.data_file = data_file
24             self.tasks = []
25             self.load()
26
27         def load(self):
```

C: > Users > lenovo > Desktop > 3-2 > AIAC > Code 9.py > TodoManager

```
21     class TodoManager:
22         def __init__(self, data_file=DATA_FILE):
23             self.data_file = data_file
24             self.tasks = []
25             self.load()
26
27         def load(self):
28             if os.path.exists(self.data_file):
29                 with open(self.data_file, "r") as f:
30                     tasks = json.load(f)
31                     self.tasks = [Task.from_dict(task) for task in tasks]
32             else:
33                 self.tasks = []
34
35         def save(self):
36             with open(self.data_file, "w") as f:
37                 json.dump([task.to_dict() for task in self.tasks], f)
38
39         def add_task(self, text):
40             text = text.strip()
41             if not text:
42                 raise ValueError("Task text cannot be empty.")
43             self.tasks.append(Task(text))
44             self.save()
45
46         def view_tasks(self, show_completed=True):
47             filtered = self.tasks if show_completed else [t for t in self.tasks if not t.done]
48             if not filtered:
49                 print("No tasks found with current filter.")
50                 return
51             for idx, task in enumerate(filtered, 1):
52                 status = "✓" if task.done else " "
53                 print(f"{idx}. [{status}] {task.text} (created {task.created_at.split('T')[0]})")
54             print(f"Total {len(filtered)} tasks.")
55
56         def delete_task(self, index, show_completed=True):
57             filtered = self.tasks if show_completed else [t for t in self.tasks if not t.done]
58             if index < 1 or index > len(filtered):
59                 raise IndexError("Task number out of range.")
60             task = filtered[index-1]
61             self.tasks.remove(task)
62             self.save()
```

C: > Users > lenovo > Desktop > 3-2 > AIAC > Code 9.py > TodoManager

```
102 def main():
124     elif choice == "5":
125         manager.view_tasks(show_completed=True)
126         idx = prompt_int("Toggle complete task number: ")
127         manager.toggle_complete(idx)
128         print("Toggled completion status.")
129     elif choice == "6":
130         if input("Are you sure? (y/n): ").strip().lower() == "y":
131             manager.clear_all()
132             print("Cleared all tasks.")
133     elif choice == "7":
134         print("Bye.")
135         break
136     else:
137         print("Invalid option.")
138 except ValueError as e:
139     print("Error:", e)
140 except IndexError as e:
141     print("Error:", e)
142 except KeyboardInterrupt:
143     print("\nInterrupted. Saving and exiting.")
144     break
145
146 if __name__ == "__main__":
147     main()
```

1. Add task

2. View tasks

3. Delete task

4. Quit

Choose an option: 1

Enter a task to add: complete project work

Added: complete project work

=== To-Do List Manager ===

1. Add task

2. View tasks

3. Delete task

4. Quit

Choose an option: █

Comparison & Analysis

Aspect	Original Code	Improved Code
Functionality	Basic features	Advanced features added
Error Handling	Minimal	Robust handling

Aspect	Original Code	Improved Code
Code Structure	Simple	Optimized and modular
User Experience	Basic	Improved

Role of AI in Collaboration

AI helped in:

- Quickly generating base code
- Identifying improvements
- Enhancing code quality and efficiency
- Acting as a virtual pair programmer

Task 2 – Version Control Integration

Objective

To use Git for managing and tracking changes in the project.

Task 2 – Version Control Integration (Single Prompt)

Act as a Git and software development expert.

1. Provide step-by-step instructions with commands to:
 - Initialize a Git repository
 - Create and switch to a new branch
 - Add AI-assisted code modifications
 - Commit with meaningful messages
 - Merge changes back into the main branch
2. Explain each step briefly.
3. Then, reflect on how Git supports AI-assisted development by discussing:
 - Benefits of branching
 - Tracking AI-generated changes
 - Collaboration advantages
 - How version history helps prevent mistakes

Git Commands Used

Initialize repository

```
git init
```

Add files

```
git add .
```

Initial commit

```
git commit -m "Initial commit"
```

Create new branch

```
git checkout -b ai-feature
```

Add AI improvements

```
git add .
```

```
git commit -m "Added AI improvements"
```

Switch branch

```
git checkout main # or master
```

Merge changes

```
git merge ai-feature
```

Screenshots

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
C:\Users\lenovo\Desktop\3-2\AIAC>git checkout -b ai-feature
Switched to a new branch 'ai-feature'

C:\Users\lenovo\Desktop\3-2\AIAC> git add Code 9.py
fatal: pathspec 'Code' did not match any files

C:\Users\lenovo\Desktop\3-2\AIAC>git add Code 9.py
fatal: pathspec 'Code' did not match any files

C:\Users\lenovo\Desktop\3-2\AIAC>git add "Code 9.py"

C:\Users\lenovo\Desktop\3-2\AIAC>git commit -m "feat: add robust todo manager with persistence and safe input handling"
[ai-feature (root-commit) b591e73] feat: add robust todo manager with persistence and safe input handling
1 file changed, 147 insertions(+)
create mode 100644 Code 9.py

C:\Users\lenovo\Desktop\3-2\AIAC>git checkout main
error: pathspec 'main' did not match any file(s) known to git

C:\Users\lenovo\Desktop\3-2\AIAC>git checkout master
error: pathspec 'master' did not match any file(s) known to git

C:\Users\lenovo\Desktop\3-2\AIAC>git add .
warning: in the working copy of 'AI Multi-Disaster Drone Simulation/Swarm_AI.ipynb', LF will be replaced by CRLF the next time Git touches it

C:\Users\lenovo\Desktop\3-2\AIAC>git commit -m "Initial commit"
[ai-feature 245280d] Initial commit
16 files changed, 396019 insertions(+)
create mode 100644 400a1a38-1bc4-499e-a875-d04ff5f3c2bf.pdf
create mode 100644 AI Multi-Disaster Drone Simulation/Swarm_AI.ipynb
create mode 100644 AI Multi-Disaster Drone Simulation/disaster map.png
create mode 100644 AI Multi-Disaster Drone Simulation/drone.jpg
create mode 100644 AI Multi-Disaster Drone Simulation/simulation.py
create mode 100644 AIAC Code.py
create mode 100644 Assignment_1.docx
create mode 100644 Code 8.py
create mode 100644 Code1.py
create mode 100644 Code2.py
create mode 100644 Code3.py
create mode 100644 Code4.py
create mode 100644 Code5.py
create mode 100644 Code6.py
create mode 100644 Code7.py
create mode 100644 merge_sort_students.py

C:\Users\lenovo\Desktop\3-2\AIAC>git branch
* ai-feature

C:\Users\lenovo\Desktop\3-2\AIAC>git checkout master
error: pathspec 'master' did not match any file(s) known to git

C:\Users\lenovo\Desktop\3-2\AIAC>git checkout ai-feature
Already on 'ai-feature'

C:\Users\lenovo\Desktop\3-2\AIAC>git branch -m master main
fatal: no branch named 'master'
```

Reflection

Version control helps in:

- Tracking AI-generated changes
- Managing different versions of code

- Enabling safe experimentation using branches
- Supporting collaboration and teamwork

Task 3 – AI for Commit Message Generation

Objective

To evaluate AI's role in generating meaningful commit messages.

Task 3 – AI for Commit Message Generation (Single Prompt)

Act as a software engineering assistant.

1. Based on the following code changes:

Generate:

- 3 concise and meaningful Git commit messages
- Follow best practices (clear, short, action-oriented)

2. Then, write 1 human-style commit message for comparison.

3. Compare AI-generated vs human-written messages in terms of:

- Clarity
- Consistency
- Time efficiency

4. Finally, explain how AI-generated commit messages improve:

- Documentation
- Traceability
- Team collaboration

Include practical examples.

AI-Generated Commit Messages

- "Added error handling to To-Do List application"
- "Refactored code for better readability and structure"
- "Implemented additional features for task management"

```
C:\Users\lenovo\Desktop\3-2\AIAC>git commit -m "feat(todo): add JSON persistence to task storage"
On branch ai-feature

C:\Users\lenovo\Desktop\3-2\AIAC>git commit -m "fix(todo): validate task input and handle invalid delete indexes"
On branch ai-feature
```

Human-Written Commit Message

- "Improved To-Do app with new features and better input handling"

```
C:\Users\lenovo\Desktop\3-2\AIAC>git commit -m "Improve to-do manager by persisting tasks, strengthening input validation, and refactoring core logic to classes"
On branch ai-feature
```

Comparison

Criteria	AI Messages	Human Message
Clarity	High	High
Consistency	Very consistent	May vary
Speed	Instant	Takes time

Analysis

AI-generated commit messages:

- Save time
- Maintain consistency
- Improve documentation quality
- Help in better traceability of changes

Task 4 – Pair Programming Simulation

Objective

To simulate collaborative coding using AI assistance.

Task 4 – Pair Programming Simulation (Single Prompt)

Act as two students collaborating on a coding project using AI.

1. Student A:

- Write a basic version of a program (e.g., To-Do List / Calculator / Expense Tracker)
- Keep it simple with core functionality and clean code

2. Student B (with AI assistance):

- Improve the program by:
 - Adding features
 - Enhancing structure
 - Improving user experience
 - Adding error handling
- Provide the updated code

3. Simulate collaboration using Git by explaining:

- Branch creation for each student
- How they worked separately
- How they merged changes
- Any conflict resolution (if applicable)

Student A – Initial Code

```
Code 9.py # student_a_todo.py Untitled-1
1 # student_a_todo.py
2 tasks = []
3
4 def add_task():
5     task = input("Enter task: ").strip()
6     if task:
7         tasks.append(task)
8         print("Added:", task)
9     else:
10        print("Task cannot be empty.")
11
12 def view_tasks():
13     if not tasks:
14         print("No tasks yet.")
15         return
16     print("Tasks:")
17     for i, t in enumerate(tasks, 1):
18         print(f"{i}. {t}")
19
20 def delete_task():
21     view_tasks()
22     if not tasks:
23         return
24     idx = input("Task number to delete: ").strip()
25     if idx.isdigit() and 1 <= int(idx) <= len(tasks):
26         removed = tasks.pop(int(idx) - 1)
27         print("Removed:", removed)
28     else:
29         print("Invalid number.")
30
31 def main():
32     while True:
33         print("\n1=Add 2=View 3=Delete 4=Quit")
34         choice = input("> ").strip()
35         if choice == "1":
36             add_task()
37         elif choice == "2":
38             view_tasks()
39         elif choice == "3":
40             delete_task()
41         elif choice == "4":
42             break
43
44 if __name__ == "__main__":
45     main()
```

0 0 0 Update is ready, click to restart. Ln 48, Col 11 Spaces: 4 U

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python + v [Icons] [Close]

3. Delete task
4. Quit
Choose an option: 1
Enter a task to add: AIAC Assignment
Added: AIAC Assignment

=== To-Do List Manager ===
1. Add task
2. View tasks
3. Delete task
4. Quit
Choose an option: 1
Enter a task to add: HPC Assignment
Added: HPC Assignment

=== To-Do List Manager ===
1. Add task
2. View tasks
3. Delete task
4. Quit
Choose an option: 2

Your tasks:
1. complete work
2. complete project work
3. AIAC Assignment
4. HPC Assignment

=== To-Do List Manager ===
1. Add task
2. View tasks
3. Delete task
4. Quit
Choose an option:

Student B – AI-Assisted Improvements

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
C: > Users > lenovo > Desktop > 3-2 > AIAC > Code 10.py > Task > to_dict
101
102 def main():
103     todo = TodoManager()
104     try:
105         while True:
106             print_menu()
107             choice = input("Option: ").strip()
108             try:
109                 if choice == "1":
110                     todo.add_task(input("Task text: "))
111                 elif choice == "2":
112                     todo.view_tasks(True)
113                 elif choice == "3":
114                     todo.view_tasks(False)
115                 elif choice == "4":
116                     todo.view_tasks(True)
117                     todo.delete_task(prompt_int("Delete #:"), True)
118                 elif choice == "5":
119                     todo.view_tasks(True)
120                     todo.toggle_task(prompt_int("Toggle #:"), True)
121                 elif choice == "6":
122                     todo.view_tasks(True)
123                     todo.edit_task(prompt_int("Edit #:"), input("New text: "), True)
124                 elif choice == "7":
125                     if input("Confirm clear all? (y/n): ").lower() == "y":
126                         todo.tasks = []
127                         todo.save()
128                         print("Cleared.")
129                 elif choice == "8":
130                     break
131             except (ValueError, IndexError) as e:
132                 print("Invalid option.")
133
Choose an option: 1
Enter a task to add: arrays topic revision
Added: arrays topic revision

=== To-Do List Manager ===
1. Add task
2. View tasks
3. Delete task
4. Quit
Choose an option: 2

Your tasks:
1. complete work
2. complete project work
3. AIAC Assignment
4. HPC Assignment
5. arrays topic revision
```

Collaboration Using Git

- Student A created the base program
- Student B created a new branch (ai-feature)
- Improvements were added using AI
- Changes were committed and merged into main branch

Outcome

- Efficient collaboration

- Better code quality
- Faster development

Final Conclusion

This assignment demonstrated the combination of AI and version control systems in modern software development.

Key learnings:

- AI can act as an effective coding assistant
- It improves productivity and code quality
- Git ensures proper tracking and collaboration
- Combining AI with Git enhances the development workflow

Overall, AI-assisted programming along with version control creates a more efficient, collaborative, and structured development process.